
Übung aus CHD2 - Chip-Design: Simulation

Thomas Müller-Wipperfürth, SS 2026

Homework 3: Single-Cycle-CPU

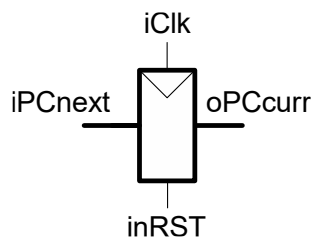
Abgabe elektronisch bis: 3. Juni 2026, 23:55 Uhr,

Bitte beachte für die Abgabe UNBEDINGT:

1. Im Header jeder abzugebenden VHDL-Datei sind Name und Matrikelnummer anzugeben.
 2. Nur **syntaktisch fehlerfreie Dateien** (VHDL-93 bzw. VHDL2002) dürfen abgegeben werden.
 3. Für das Modell der Single-Cycle-CPU wird ein Package `SingleCycleCPUPack` verwendet, das in der Datei `SCCPU-p.vhd` definiert ist. Diese Datei enthält alle Datentypen und Konstanten, die in den *entity*- und *architecture*-Beschreibungen der CPU Verwendung finden.
 4. **Vordefinierte Typen und Konstanten sind unbedingt zu verwenden. Das package `SingleCycleCPUPack` ist bei Bedarf geeignet zu erweitern!**
 5. Abgabe: Die abzugebenden VHDL- und PDF-Dateien sind in ein Verzeichnis `Homework3` (ohne Unterverzeichnisse) zu legen. Dieses Verzeichnis ist in ein ZIP-Archiv zu packen. Als Name der Zip-Datei ist `<Familiename>_HW3.zip` zu wählen. Die Abgabe erfolgt über das Elearning-System.
-

Aufgabe 1 – Program Counter

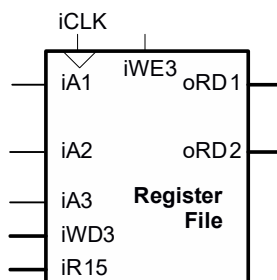
Modelliere die *entity* `PC` mit einer *architecture* `Bhv` für den *Program Counter* der Single-Cycle-CPU. Erstelle dazu die Datei `PC-ea.vhd`. Die Portnamen sind in der nachstehenden Graphik abgebildet.



Die Ports `iClk` und `inRST` Ports sind vom Typ `std_ulogic`. `iPCnext` und `oPCcurr` sind als der vordefinierte Typ `aPCvalue` – ein *subtype* von `std_ulogic_vector` – zu definieren (findet sich im package `SingleCycleCPUPack`). Teste die *entity/architecture* von `PC` durch Modifikation der Testbench des D-Flipflops aus der Übung 5. Verwende und ergänze dazu die Datei `PC-tb.vhd`.

Aufgabe 2 – Registerfile (doppelte Bewertung)

Modelliere das Register-File `RegFile` der Single-Cycle-CPU als *entity* mit der *architecture* `Bhv`. Das Verhalten des Register-Files soll durch zwei *processes* in der *architecture* `Bhv` realisiert werden:



ReadReg:

Dieser kombinatorische *process* wartet auf Events und gibt den intern gespeicherten Wert der adressierten Register an den Ports `oRD1` und `oRD2` aus. Wird die Registeradresse 15 (= Programm-Counter) angelegt, ist der Wert des Inputports `iR15` am entsprechenden Datenport auszugeben.

Der zweite *process* in der *architecture* `Bhv` ist der synchrone *process* `WriteReg`. Dieser *process* ist wie das D-Flipflop auf das Taktsignal `iCLK` sensitiv. D.h., nur bei einer steigenden Flanke am Taktsignal `iCLK` wird der *process* aktiv. Ist zusätzlich der Wert des Inputports `WE3` auf '1' gesetzt, so wird der Wert am Inputport `WD3` in jenes Register geschrieben, das durch `iA3` adressiert wird. Achtung: Der PC kann hierdurch nicht beschrieben werden.

Ein *Reset*, d.h., ein Nullsetzen aller Register ist nicht zu modellieren!

Hinweis 1: Überlege eine interne Datenstruktur zum Abspeichern der Registerwerte. Der Zugriff auf ein Register erfolgt über eine Dualzahl (repräsentiert in einem `std_ulogic_vector`). Im package `SingleCycleCPUPack` ist eine *function* zur Konvertierung in eine Zahl vom Typ `natural` bereits vordefiniert.

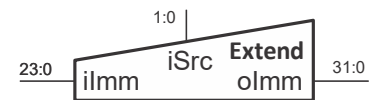
Erstelle eine Testbench in der Datei `RegFile-tb.vhd` und teste die *entity/architecture* des Register-Files durch gut überlegte Testdaten. Die Einhaltung der Setup-Zeit ist auch hier nicht zu überprüfen.

Teste, wie sich das Registerfile im folgenden Fall verhält:

- Mit den Inputports `iA1` und `iA3` wird dasselbe Register (z.B. `r1`) adressiert.
- Am Port `WD3` liegt ein neuer - ungleicher - Wert für dieses Register an.
- Der Port `iWE3` ist mit '1' belegt.
- Am Takteingang `iCLK` wird eine steigende Flanke erkannt
- Welchen Wert hat der Outputport `oRD1` nach Ablauf aller Verzögerungszeiten?

Aufgabe 3 – Sign Extension Unit

Modelliere und teste die *entity* `Extend` mit einer *architecture* `Bhv` für die Single-Cycle-CPU (Datei: `Extend-ea.vhd`). Die Portnamen sind in der nebenstehenden Graphik abgebildet – die Funktionalität ist den COD2-Vorlesungsunterlagen zu entnehmen. An den Port `iSrc` legt der Controller die Bits [27:26] der Instruktion an. Benutze folgende Deklarationen (siehe `SCCPU-p.vhd`):



```
subtype aOpCode is std_ulogic_vector (0 to 1);
constant cDOpCode : aOpCode := "00"; -- data process.
constant cMOpCode : aOpCode := "01"; -- memory
constant cBOpCode : aOpCode := "10"; -- branch
```

Abgabe:

- `SCCPU-p.vhd`
- `PC-ea.vhd`, `PC-tb.vhd`
- `RegFile-ea.vhd`, `RegFile-tb.vhd`
- `Extend-ea.vhd`

Alle VHDL-Dateien sind auch als PDF-Dateien abzugeben.

Die abzugebenden Dateien sind in ein Verzeichnis `Homework3` (ohne Unterverzeichnisse) zu legen. Dieses Verzeichnis ist in ein ZIP-Archiv zu packen. Als Name der Zip-Datei ist `<Familiennamen>_HW3.zip` zu wählen. Die Abgabe erfolgt über das Elearning-System.